the type of the variable to be the same as the initializer. Thus, the iterator example can be written as

```
auto iter=m.find("my key");
```

Now, you're not restricted to just plain `auto`; you can embellish it to declare `const` variables or pointer or reference variables too. Here are a few variable declarations using `auto` and the corresponding type of the variable:

```
auto i=42;        // int
auto& j=i;        // int&
auto const k=i;   // int const
auto* const p=&i; // int * const
```

The rules for deducing the type of the variable are based on the rules for the only other place in the language where types are deduced: parameters of function templates. In a declaration of the form

```
some-type-expression-involving-auto var=some-expression;
```

the type of `var` is the same as the type deduced for the parameter of a function template declared with the same type expression, except replacing `auto` with the name of a template type parameter:

```
template<typename T>
void f(type-expression var);
f(some-expression);
```

This means that array types decay to pointers, and references are dropped unless the type expression explicitly declares the variable as a reference, for example:

```
int some_array[45];
auto p=some_array;    // int*
int& r=*p;
auto x=r;             // int
auto& y=r;            // int&
```

This can greatly simplify the declaration of variables, particularly where the full type identifier is long or possibly not even known (for example, the type of the result of a function call in a template).